Methods

Methods I

The `template/` architecture is deliberately bifurcated into a globally shared `infrastructure/` layer and project-specific `projects/` silos. This section describes the four core design patterns, the YAML-declared pipeline (17 stages; default full 10) pipeline that operationalizes them, and the AI collaboration model that distinguishes this system from conventional research templates.

The Two-Layer Architecture I

The repository is organized into two strictly separated layers:

Infrastructure Layer (`infrastructure/`): 31 infrastructure subdirectories—28 of them independently-importable Python packages—comprising ~1334 modules and providing reusable services. Each importable package has its own `__init__.py`, `AGENTS.md`, and `README.md`, and exports a well-defined public API (the remaining subdirectories, e.g. `config/`, hold shared configuration). The infrastructure layer knows nothing about any specific project—it provides generic capabilities (logging, rendering, validation, steganography) that any project may consume.

Project Layer (`projects/`): Self-contained research workspaces. Each project directory contains:

The Two-Layer Architecture I

Directory	Purpose
`manuscript/`	Markdown chapters and `config.yaml`
`scripts/`	Thin orchestrator scripts (Stage 02)
`src/`	Project-specific Python modules
`tests/`	Project-specific test suite
`data/`	Input datasets and generated data
`output/`	Pipeline artifacts: PDF, figures, reports, logs
`docs/`	Project-specific architecture documentation

The Two-Layer Architecture I

The two layers communicate exclusively through Python imports and filesystem paths. No project modifies infrastructure code; no infrastructure module references a specific project by name (except via runtime project discovery).

The Standalone Project Paradigm I

Projects are designed to be completely self-contained. Adding a new project requires no changes to the infrastructure layer, no modifications to `pyproject.toml`, and no updates to the pipeline orchestrator. A project is automatically discovered if and only if it satisfies two conditions:

The Standalone Project Paradigm I

1. It exists as a subdirectory of `projects/`.
2. It contains the file `manuscript/config.yaml`.

This paradigm enables horizontal scaling: N researchers can maintain N independent projects within a single repository, sharing infrastructure without coupling. Each project declares its own testing tolerances, manuscript metadata, LLM review preferences, and rendering configuration in its `config.yaml`. The system currently hosts its public canonical exemplars under `projects/templates/` (`templates/template_active_inference`, `templates/template_advanced_literature_review`, `templates/template_autopoiesis`, `templates/template_autoresearch_project`, `templates/template_autoscientists`, `templates/template_code_project`, `templates/template_data_descriptor`, `templates/template_eda_notebook`, `templates/template_experiment_tree`, `templates/template_formal`, `templates/template_gold_refinement`,

templates/template_literature_meta_analysis, templates/template_madlib, templates/template_methods_paper, templates/template_newspaper, templates/template_pitch_deck, templates/template_pools_rules_tools, templates/template_prose_project, templates/template_redacted_report, templates/template_registered_report, templates/template_search_project, templates/template_sia, templates/template_storybook, templates/template_template, templates/template_textbook), including this meta-manuscript at projects/templates/template_template/.

The Thin Orchestrator Pattern I

All scripts in `scripts/` (both infrastructure-level and project-level) follow the Thin Orchestrator pattern (Gamma et al. 1995):

The Thin Orchestrator Pattern I

- **No domain logic**: Scripts contain zero algorithmic implementation. They import functions from `src/` modules and wire them to infrastructure services.
- **Configuration-driven**: Behavior is parameterized by `config.yaml`, not by hardcoded values.
- **Stateless**: Scripts read inputs, call functions, write outputs. They maintain no persistent state between invocations.
- **Logged**: Every significant action is logged via `infrastructure.core.logging.utils.get_logger`.

The Thin Orchestrator Pattern I

This pattern ensures that all testable logic lives in `src/` where it is subject to the Zero-Mock testing policy, while scripts remain thin enough to be audited by visual inspection. The separation draws on the Mediator pattern from Gamma et al. (Gamma et al. 1995), where scripts mediate between infrastructure services and project-specific code without implementing any logic of their own.

To make this concrete, the following contrasts the anti-pattern with the correct pattern:

The Thin Orchestrator Pattern I

```python
# ANTI-PATTERN: domain logic embedded in script
def calculate_average(data):                    # ← never put
computation here
    return sum(data) / len(data)

result = calculate_average([1, 2, 3])

# CORRECT PATTERN: script imports from src/ and only
wires
from projects.my_project.src.statistics import
calculate_average

result = calculate_average([1, 2, 3])  # ← scripts
wire, never compute
```

The Thin Orchestrator Pattern I

The critical property is that `calculate_average` in the correct pattern lives in a testable `src/` module, is covered by the Zero-Mock test suite, and can be independently imported, tested, and reused—whereas the anti-pattern buries logic in a script that is invisible to coverage tools.